sorting arrays in numpy by column

How can I sort an array in numpy by the nth column? e.g.

```
a = array([[1,2,3],[4,5,6],[0,0,1]])
```

I'd like to sort by the second column, such that I get back:

```
array([[0,0,1],[1,2,3],[4,5,6]])
```

thanks.

python sorting numpy scipy

asked May 13 '10 at 15:32

user248237dfsf

11.5k 54 189 305

5 Answers

@steve's is actually the most elegant way of doing it.

For the "correct" way see the order keyword argument of numpy.ndarray.sort

However, you'll need to view your array as an array with fields (a structured array).

The "correct" way is quite ugly if you didn't initially define your array with fields...

As a quick example, to sort it and return a copy:

```
In [1]: import numpy as np

In [2]: a = np.array([[1,2,3],[4,5,6],[0,0,1]])

In [3]: np.sort(a.view('i8,i8,i8'), order=['f1'], axis=0).view(np.int)
Out[3]:
array([[0, 0, 1],
       [1, 2, 3],
       [4, 5, 6]])
```

To sort it in-place:

```
In [6]: a.view('i8,i8,i8').sort(order=['f1'], axis=0) #<-- returns None

In [7]: a
Out[7]:
array([[0, 0, 1],
       [1, 2, 3],
       [4, 5, 6]])
```

@Steve's really is the most elegant way to do it, as far as I know...

The only advantage to this method is that the "order" argument is a list of the fields to order the search by. For example, you can sort by the second column, then the third column, then the first column by supplying order=['f1','f2','f0'].

edited May 13 '10 at 16:18

answered May 13 '10 at 16:10

Joe Kington

106k 16 237 269

2 In my numpy 1.6.1rc1, it raises `ValueError: new type not compatible with array.` – Clippit Oct 5 '11 at 17:40

2 That example assumes you're on a 64-bit system. If you're not, try replacing `'i8,i8,i8'` with `'i4,i4,i4'` . – Joe Kington Oct 5 '11 at 18:47

5 Would it make sense to file a feature request that the "correct" way be made less ugly? – endolith Aug 21 '13 at 3:15

1 And for hybrid type like `a = np.array([['a',1,2,3],['b',4,5,6],['c',0,0,1]])` what approach should I follow? – MrMartin May 9 '15 at 16:50

1 @MrMartin - Based on this and some of your other comments, it sounds like you have data with a common type in each column (e.g. spreadsheet-like data). If so, have a look at `pandas` . It will simplify a lot of the type of operations you're wanting to do. – Joe Kington May 9 '15 at 20:16

I suppose this works: `a[a[:,1].argsort()]`

edited Sep 28 '11 at 20:12
EOL
35.3k 19 108 164

answered May 13 '10 at 15:39
Steve Tjoa
21.4k 6 52 78

Looks ugly, though. I would also like to know a better way. – Steve Tjoa May 13 '10 at 15:40

This is not clear, what is `1` in here? the index to be sorted by? – emab Apr 14 '14 at 5:30

4 `[:,1]` indicates the second column of `a` . – Steve Tjoa Apr 17 '14 at 20:49

1 should have been accepted as the right answer and it is faster. – hashmuke Dec 25 '14 at 15:04

5 If you want the reverse sort, modify this to be `a[a[:,1].argsort()[::-1]]` – stvn66 May 14 '15 at 14:49

From the python docs wiki link, I think you can do :

```
a = ([[1,2,3],[4,5,6],[0,0,1]]);
a = sorted(a, key=lambda a_entry: a_entry[1])
print a
```

Output is:

```
[[[0, 0, 1], [1, 2, 3], [4, 5, 6]]]
```

edited Sep 28 '11 at 20:23

answered Sep 28 '11 at 20:05
user541064
154 1 6

9 With this solution, one gets a list instead of a NumPy array, so this might not always be convenient (takes more memory, is probably slower, etc.). – EOL Sep 28 '11 at 20:13

3 Oh, ok. I totally missed that point. Thanks! – user541064 Sep 28 '11 at 20:22

From the numpy mailing list, here's another solution:

```
>>> a
array([[1, 2],
       [0, 0],
       [1, 0],
       [0, 2],
       [2, 1],
       [1, 0],
       [1, 0],
       [0, 0],
       [1, 0],
      [2, 2]])
>>> a[np.lexsort(np.fliplr(a).T)]
array([[0, 0],
       [0, 0],
       [0, 2],
       [1, 0],
       [1, 0],
       [1, 0],
       [1, 0],
       [1, 2],
       [2, 1],
       [2, 2]])
```

answered Jun 3 '15 at 15:03
fgregg
927 7 20

In case someone wants to make use of sorting at a critical part of their programs here's a

performance comparison for the different proposals:

```
import numpy as np
table = np.random.rand(5000, 10)

%timeit table.view('f8,f8,f8,f8,f8,f8,f8,f8,f8,f8').sort(order=['f9'], axis=0)
1000 loops, best of 3: 1.88 ms per loop

%timeit table[table[:,9].argsort()]
10000 loops, best of 3: 180 µs per loop

import pandas as pd
df = pd.DataFrame(table)
%timeit df.sort_values(9, ascending=True)
1000 loops, best of 3: 400 µs per loop
```

So, it looks like indexing with argsort is the quickest method so far...

answered Feb 25 at 10:37

MonkeyButter
542 4 14